

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I V. Sujithra (192521445) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Question 1

Analyze the case:

An online shopping platform stores:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Identify the **Functional Dependencies (FDs)** and normalize the relation up to **Third Normal Form (3NF)**.

Introduction

An online shopping platform maintains information about customers, products, and orders in a single relation. Although storing all details in one table is simple, it leads to redundancy, update anomalies, insertion anomalies, and deletion anomalies. Database normalization is used to organize the data efficiently by reducing redundancy and improving data integrity.

The given relation is:

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Attribute Description

- OrderID – Unique identifier for each order.
- CustID – Unique customer ID.
- CustName – Name of the customer.
- ProdID – Unique product ID.
- ProdName – Name of the product.
- Qty – Quantity ordered.
- Price – Price of the product.

Step 1: Identify Functional Dependencies (FDs)

From the given relation, the following functional dependencies can be identified:

1. **OrderID $\rightarrow$ CustID**
2. **CustID $\rightarrow$ CustName**
3. **ProdID $\rightarrow$ ProdName, Price**
4. **(OrderID, ProdID) $\rightarrow$ Qty**

These dependencies indicate how one attribute determines another.

Step 2: Candidate Key

Since an order may contain multiple products, neither OrderID nor ProdID alone can uniquely identify every record.

Therefore,

Candidate Key = (OrderID, ProdID)

This composite key uniquely identifies each tuple.

Step 3: Problems in the Relation

The current table suffers from several anomalies.

1. Data Redundancy

Customer name is repeated for every product ordered by the same customer.

Product name and price are repeated in every order.

This wastes storage space.

2. Update Anomaly

If the customer's name changes, every record must be updated.

If one record is missed, inconsistent data occurs.

Similarly, changing the price requires updating multiple rows.

3. Insertion Anomaly

A new customer cannot be added unless an order is placed.

A new product cannot be inserted without creating an order.

4. Deletion Anomaly

Deleting the last order of a customer removes customer information.

Deleting the last sale of a product removes product details.

Normalization

First Normal Form (1NF)

A relation is in 1NF if:

- Every attribute contains only atomic values.
- No repeating groups exist.

The given relation already satisfies these conditions.

Therefore,

Relation in 1NF

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Second Normal Form (2NF)

A relation is in 2NF when:

- It is in 1NF.
- Every non-key attribute is fully dependent on the whole primary key.

The primary key is

(OrderID, ProdID)

However,

CustID depends only on OrderID.

CustName depends on CustID.

ProdName and Price depend only on ProdID.

Therefore, partial dependency exists.

Hence the relation is **not in 2NF**.

Decomposition into 2NF

ORDER

Order(OrderID, CustID)

Primary Key: OrderID

CUSTOMER

Customer(CustID, CustName)

Primary Key: CustID

PRODUCT

Product(ProdID, ProdName, Price)

Primary Key: ProdID

ORDER_DETAILS

Order_Details(OrderID, ProdID, Qty)

Primary Key:

(OrderID, ProdID)

Foreign Keys:

OrderID references ORDER

ProdID references PRODUCT

Now every non-key attribute depends completely on its primary key.

Hence the database is in **2NF**.

Third Normal Form (3NF)

A relation is in 3NF when:

- It is already in 2NF.
- No transitive dependency exists.

Previously,

$\text{OrderID} \rightarrow \text{CustID} \rightarrow \text{CustName}$

This is a transitive dependency.

By placing customer information into a separate CUSTOMER table, the transitive dependency is removed.

Thus all relations satisfy 3NF.

Final 3NF Relations

ORDER

Relation: ORDER(OrderID, CustID)

- **OrderID** – Primary Key
- **CustID** – Foreign Key (References CUSTOMER)

CUSTOMER

Relation: CUSTOMER(CustID, CustName)

- **CustID** – Primary Key
- **CustName** – Customer Name

PRODUCT

Relation: PRODUCT(ProdID, ProdName, Price)

- **ProdID** – Primary Key
- **ProdName** – Product Name
- **Price** – Product Price

ORDER_DETAILS

Relation: ORDER_DETAILS(OrderID, ProdID, Qty)

- **OrderID** – Foreign Key (References ORDER)
- **ProdID** – Foreign Key (References PRODUCT)
- **Qty** – Quantity Ordered

Primary Key: (*OrderID, ProdID*)

Advantages of 3NF

- Eliminates duplicate data.
- Removes insertion anomalies.
- Removes deletion anomalies.
- Prevents update anomalies.
- Improves consistency.
- Reduces storage requirements.
- Makes database maintenance easier.
- Improves overall database integrity.
- Supports efficient data retrieval.
- Provides a structured database design.

Conclusion

The given online shopping relation contains both partial and transitive dependencies. By identifying the functional dependencies and decomposing the relation into **ORDER**, **CUSTOMER**, **PRODUCT**, and **ORDER_DETAILS**, the database is successfully normalized up to **Third Normal Form (3NF)**. The normalized schema minimizes redundancy, avoids anomalies, and ensures data consistency and integrity.

Question 2

Analyze the Case:

A hospital maintains the relation:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Identify the anomalies and normalize the relation up to **Boyce-Codd Normal Form (BCNF)**.

Introduction

Hospitals maintain records of patients, doctors, and wards for efficient healthcare management. In the given relation, all information is stored in a single table. Although this makes data retrieval simple, it introduces redundancy and anomalies. Database normalization helps eliminate these problems by organizing data into multiple related tables.

The given relation is:

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Attribute Description

- **PID** – Patient ID
- **PName** – Patient Name
- **DocID** – Doctor ID
- **DocName** – Doctor Name
- **Specialization** – Doctor's specialization
- **WardNo** – Ward Number
- **WardName** – Ward Name
-

Step 1: Functional Dependencies

The following functional dependencies are identified:

1. **PID $\rightarrow$ PName, DocID, WardNo**
2. **DocID $\rightarrow$ DocName, Specialization**
3. **WardNo $\rightarrow$ WardName**

From these dependencies:

- Patient ID determines patient details.
- Doctor ID determines doctor information.
- Ward Number determines ward name.

Step 2: Candidate Key

The only attribute that uniquely identifies each patient record is:

Candidate Key = PID

Therefore,

Primary Key = PID

Step 3: Identify Anomalies

Since all information is stored in one table, several anomalies occur.

1. Data Redundancy

Doctor information is repeated for every patient treated by the same doctor.

Ward name is repeated for every patient admitted to the same ward.

This increases unnecessary storage.

2. Update Anomaly

If a doctor's specialization changes, every related record must be updated.

If one record is missed, inconsistent information appears.

Similarly, changing a ward name requires updating multiple rows.

3. Insertion Anomaly

A new doctor cannot be added unless a patient is assigned.

A new ward cannot be recorded without admitting a patient.

4. Deletion Anomaly

Deleting the last patient of a doctor removes the doctor's information.

Deleting the last patient from a ward removes ward details.

Normalization

First Normal Form (1NF)

A relation is in 1NF if:

- All values are atomic.
- No repeating groups exist.

The given relation already satisfies these conditions.

Therefore,

Relation is in 1NF.

Second Normal Form (2NF)

The primary key is **PID**, which consists of a single attribute.

Since there is no composite key, there are no partial dependencies.

Hence,

Relation is already in 2NF.

Third Normal Form (3NF)

A relation is in 3NF if there are no transitive dependencies.

The following transitive dependencies exist:

- **$PID \rightarrow DocID \rightarrow DocName, Specialization$**
- **$PID \rightarrow WardNo \rightarrow WardName$**

These dependencies violate 3NF.

Decomposition into 3NF

PATIENT

Patient(PID, PName, DocID, WardNo)

Primary Key: PI

DOCTOR

Doctor(DocID, DocName, Specialization)

Primary Key: DocID

WARD

Ward(WardNo, WardName)

Primary Key: WardNo

Now all transitive dependencies are removed

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if:

For every functional dependency $X \rightarrow Y$, X must be a candidate key.

Checking each table:

PATIENT

$PID \rightarrow PName, DocID, WardNo$

PID is the candidate key.

✓ BCNF satisfied.

DOCTOR

$DocID \rightarrow DocName, Specialization$

DocID is the candidate key.

✓ BCNF satisfied.

WARD

WardNo $\rightarrow$ WardName

WardNo is the candidate key.

✓ BCNF satisfied.

Therefore, all decomposed relations satisfy **BCNF**.

Final BCNF Relations

PATIENT

- PID – Primary Key
- PName
- DocID – Foreign Key
- WardNo – Foreign Key

DOCTOR

- DocID – Primary Key
- DocName
- Specialization

WARD

- WardNo – Primary Key
- WardName

Advantages of BCNF

- Eliminates data redundancy.
- Prevents update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- Ensures every determinant is a candidate key.
- Improves data consistency.
- Simplifies database maintenance.

- Provides better integrity.
- Improves query performance.
- Makes the database easier to manage.

Conclusion

The original hospital relation contains redundancy and transitive dependencies, leading to update, insertion, and deletion anomalies. By decomposing the relation into **PATIENT**, **DOCTOR**, and **WARD**, all functional dependencies are preserved, redundancy is minimized, and the database satisfies **Boyce-Codd Normal Form (BCNF)**. This design ensures efficient storage, data integrity, and easier maintenance.

Question 3

Case Study:

A university stores:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Analyze the functional dependencies and apply **Second Normal Form (2NF)**.

Introduction

A university maintains student enrollment information in a single table. This table stores student details, course information, faculty details, and grades. Although this design is simple, it contains redundant information that can cause anomalies. Applying normalization improves data consistency and reduces redundancy.

The given relation is:

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

Attribute Description

- SID – Student ID
- SName – Student Name
- CourseID – Course ID
- CourseName – Course Name
- Faculty – Faculty handling the course
- Grade – Student Grade

Functional Dependencies

1. $SID \rightarrow SName$
2. $CourseID \rightarrow CourseName, Faculty$
3. $(SID, CourseID) \rightarrow Grade$

Candidate Key

A student can enroll in many courses.

A course can have many students.

Therefore,

Candidate Key = (SID, CourseID)

Problems in the Existing Relation

Data Redundancy

- Student name is repeated for every enrolled course.
- Course name and faculty are repeated for every student.

Update Anomaly

Updating faculty information requires changing multiple rows.

Insertion Anomaly

A new course cannot be inserted until at least one student enrolls.

Deletion Anomaly

Deleting the last student enrolled in a course removes the course details.

Normalization

1NF

The relation contains only atomic values.

Hence it is already in **1NF**.

2NF

The primary key is (SID, CourseID).

Partial dependencies exist:

- $SID \rightarrow SName$
- $CourseID \rightarrow CourseName, Faculty$

To remove these dependencies, decompose the relation.

Decomposed Relations

STUDENT

Student(SID, SName)

Primary Key: SID

COURSE

Course(CourseID, CourseName, Faculty)

Primary Key: CourseID

ENROLLMENT

Enrollment(SID, CourseID, Grade)

Primary Key:

(SID, CourseID)

Foreign Keys:

SID references STUDENT

CourseID references COURSE

Advantages of 2NF

- Removes partial dependency
- Reduces redundancy
- Prevents anomalies
- Improves consistency
- Easier maintenance

Conclusion

The Enrollment relation contains partial dependencies. After decomposition into **STUDENT**, **COURSE**, and **ENROLLMENT**, the database satisfies **Second Normal Form (2NF)** and ensures better integrity and efficient storage.

Question 4

Case Study:

Given the airline booking relation:

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Normalize the relation up to **BCNF**.

Introduction

Airline reservation systems store passenger and flight information. Storing all details in one relation causes redundancy and inconsistency. BCNF removes these issues.

Functional Dependencies

1. $\text{FlightNo} \rightarrow \text{DeptCity}, \text{ArrCity}, \text{Date}$
2. $\text{PassID} \rightarrow \text{PassName}$
3. $(\text{FlightNo}, \text{PassID}) \rightarrow \text{Seat}$

Candidate Key

$(\text{FlightNo}, \text{PassID})$

Anomalies

Update

Changing departure city requires updating many rows.

Insertion

A flight cannot be added without a passenger.

Deletion

Deleting the last passenger removes flight information.

Normalization

1NF

Already satisfies atomicity.

2NF

Partial dependencies exist.

FlightNo determines flight information.

PassID determines passenger information.

3NF

Separate flight and passenger details.

BCNF Decomposition

FLIGHT

$\text{Flight}(\text{FlightNo}, \text{DeptCity}, \text{ArrCity}, \text{Date})$

Primary Key: FlightNo

PASSENGER

Passenger(PassID, PassName)

Primary Key: PassID

BOOKING

Booking(FlightNo, PassID, Seat)

Primary Key:

(FlightNo, PassID)

Foreign Keys:

FlightNo references FLIGHT

PassID references PASSENGER

Advantages

- Eliminates redundancy
- Prevents anomalies
- Improves consistency
- Better maintenance
- Efficient queries

Conclusion

The airline booking database is successfully normalized into **FLIGHT**, **PASSENGER**, and **BOOKING** relations, satisfying **BCNF**.

Question 5

Case Study:

Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize the relation to **3NF**.

Introduction

An HR system stores employee, department, and manager information. If stored in a single table, redundancy and anomalies occur. Normalization removes these issues.

Assume the relation:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, Salary)

Functional Dependencies

1. $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}, \text{Salary}$
2. $\text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$
3. $\text{ManagerID} \rightarrow \text{ManagerName}$

Candidate Key

EmpID

Transitive Dependencies

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}$

$\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{ManagerID} \rightarrow \text{ManagerName}$

These violate 3NF.

Normalization

1NF

Atomic values present.

2NF

Primary key is single attribute.

Already in 2NF.

3NF Decomposition

EMPLOYEE

Employee(EmpID, EmpName, DeptID, Salary)

Primary Key: EmpID

DEPARTMENT

Department(DeptID, DeptName, ManagerID)

Primary Key: DeptID

MANAGER

Manager(ManagerID, ManagerName)

Primary Key: ManagerID

Advantages

- Eliminates transitive dependency
- Removes redundancy

- Better consistency
- Simplifies maintenance
- Prevents anomalies

Conclusion

The HR schema contains transitive dependencies. After decomposition into **EMPLOYEE**, **DEPARTMENT**, and **MANAGER**, the database satisfies **Third Normal Form (3NF)** and provides efficient data management.

Question 6

Case Study:

A library maintains the relation:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Identify the **Multivalued Dependencies (MVDs)** and normalize the relation up to **Fourth Normal Form (4NF)**.

Introduction

A library database stores member information, book details, and borrowing records. When all data is stored in a single table, duplication occurs because one member can borrow multiple books and one book can be borrowed by different members. Fourth Normal Form (4NF) removes multivalued dependencies and minimizes redundancy.

The given relation is:

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Attribute Description

- MemberID – Unique ID of the member
- MemberName – Name of the member
- BookID – Unique ID of the book
- Title – Book title
- Author – Author of the book
- Genre – Book category
- BorrowDate – Date on which the book was borrowed

Functional Dependencies

1. $\text{MemberID} \rightarrow \text{MemberName}$
2. $\text{BookID} \rightarrow \text{Title, Author, Genre}$
3. $(\text{MemberID}, \text{BookID}) \rightarrow \text{BorrowDate}$

Multivalued Dependencies (MVDs)

The following multivalued dependencies exist:

- $\text{MemberID} \twoheadrightarrow \text{BookID}$
- $\text{BookID} \twoheadrightarrow \text{MemberID}$

This means:

- A member can borrow many books.
- A book can be borrowed by many members (over time).

Problems in the Existing Relation

Data Redundancy

- Member name is repeated for every borrowed book.
- Book details are repeated whenever different members borrow the same book.

Update Anomaly

Updating an author's name requires updating multiple rows.

Insertion Anomaly

A new member or new book cannot be added unless a borrowing record exists.

Deletion Anomaly

Deleting the last borrowing record removes either member or book details.

Normalization

1NF

All attributes contain atomic values.

Hence, the relation is already in **1NF**.

2NF

Remove partial dependencies by separating member and book details.

3NF

No transitive dependencies remain after decomposition

4NF

To remove multivalued dependencies, decompose the relation into three tables.

MEMBER

Member(MemberID, MemberName)

Primary Key: MemberID

BOOK

Book(BookID, Title, Author, Genre)

Primary Key: BookID

BORROW

Borrow(MemberID, BookID, BorrowDate)

Primary Key:

(MemberID, BookID)

Foreign Keys:

- MemberID references MEMBER
- BookID references BOOK

Advantages of 4NF

- Eliminates multivalued dependencies.
- Reduces redundancy.
- Improves data consistency.
- Simplifies maintenance.
- Prevents update, insertion, and deletion anomalies.
- Ensures efficient storage.
- Improves database performance.
- Makes future modifications easier.

Conclusion

The Library relation contains multivalued dependencies because members can borrow multiple books. By decomposing the relation into **MEMBER**, **BOOK**, and **BORROW**, the database satisfies **Fourth Normal Form (4NF)** and provides efficient, consistent data storage.

Question 7

Case Study:

Analyze a telecom billing schema and identify all **candidate keys**, **primary keys**, and **functional dependencies** before normalization.

Introduction

A telecom billing system stores customer information, mobile plans, and billing details. Proper identification of keys and functional dependencies is essential before normalization.

Assume the relation:

Billing(BillID, CustomerID, CustomerName, PlanID, PlanName, Amount, BillDate)

Attribute Description

- BillID – Bill Number
- CustomerID – Customer ID
- CustomerName – Customer Name
- PlanID – Plan ID
- PlanName – Mobile Plan
- Amount – Bill Amount
- BillDate – Billing Date

Functional Dependencies

1. BillID $\rightarrow$ CustomerID, PlanID, Amount, BillDate
2. CustomerID $\rightarrow$ CustomerName
3. PlanID $\rightarrow$ PlanName

Candidate Keys

A candidate key uniquely identifies every record.

Candidate Key:

BillID

Primary Key

Primary Key:

BillID

Problems Before Normalization

- Customer name is repeated.

- Plan details are duplicated.
- Updating customer information requires multiple updates.
- Deleting the last bill removes customer information.
- Inserting a new plan is impossible without creating a bill.

Normalization Overview

Separate the database into:

CUSTOMER

Customer(CustomerID, CustomerName)

PLAN

Plan(PlanID, PlanName)

BILL

Bill(BillID, CustomerID, PlanID, Amount, BillDate)

Advantages

- Eliminates redundancy.
- Identifies unique records correctly.
- Improves integrity.
- Simplifies maintenance.
- Supports efficient billing operations.

Conclusion

The telecom billing schema uses **BillID** as the primary and candidate key. Functional dependencies help identify redundancy and guide the normalization process for improved database efficiency.

Question 8

Case Study:

A retail inventory system has:

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Apply **complete normalization**.

Introduction

Retail inventory systems store product, supplier, and warehouse information. Storing everything in one relation leads to duplication and anomalies. Complete normalization organizes data efficiently.

Functional Dependencies

1. $PID \rightarrow PName, SupplierID, Category, Price, Warehouse$
2. $SupplierID \rightarrow SupplierName$

Candidate Key

Candidate Key:

PID

Problems

Data Redundancy

Supplier name repeats for every product.

Update Anomaly

Changing supplier information requires multiple updates.

Insertion Anomaly

A supplier cannot be inserted without adding a product.

Deletion Anomaly

Deleting the last product removes supplier information.

Normalization

1NF

Relation already contains atomic values.

2NF

Primary key is single attribute.

Already in 2NF.

3NF

Remove transitive dependency.

$SupplierID \rightarrow SupplierName$

Final Relations

PRODUCT

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key: PI

SUPPLIER

Supplier(SupplierID, SupplierName)

Primary Key: SupplierID

Advantages

- Eliminates redundancy.
- Prevents anomalies.
- Improves consistency.
- Efficient storage.
- Better maintenance.
- Faster queries.
- Easier inventory management.
- Improves database integrity.

Conclusion

The retail inventory system contains transitive dependency due to supplier details. By decomposing the relation into **PRODUCT** and **SUPPLIER**, the database becomes fully normalized, ensuring efficient and reliable inventory management.

Question 9

Case Study:

Study the given poorly normalized schema for a **School ERP** and propose a fully normalized design up to **Boyce-Codd Normal Form (BCNF)** with justification.

Introduction

A School ERP (Enterprise Resource Planning) system stores information about students, teachers, classes, and subjects. If all details are maintained in a single relation, it causes data redundancy, inconsistency, and anomalies. Normalization helps organize the database efficiently and ensures data integrity.

Assume the relation is:

School(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName)

Attribute Description

- **StudentID** – Unique student identifier
- **StudentName** – Name of the student
- **ClassID** – Class identifier
- **ClassName** – Name of the class
- **TeacherID** – Teacher identifier
- **TeacherName** – Teacher name
- **SubjectID** – Subject identifier
- **SubjectName** – Subject name

Functional Dependencies

1. $\text{StudentID} \rightarrow \text{StudentName}, \text{ClassID}$
2. $\text{ClassID} \rightarrow \text{ClassName}, \text{TeacherID}$
3. $\text{TeacherID} \rightarrow \text{TeacherName}$
4. $\text{SubjectID} \rightarrow \text{SubjectName}$

Candidate Key

The candidate key is:

StudentID

Problems in the Existing Relation

1. Data Redundancy

- Teacher name is repeated for every student.
- Subject information is repeated multiple times.
- Class details are duplicated.

2. Update Anomaly

Changing a teacher's name requires updating many records.

3. Insertion Anomaly

A new class or teacher cannot be inserted without adding a student.

4. Deletion Anomaly

Deleting the last student from a class removes class and teacher information.

Normalization Process

First Normal Form (1NF)

The relation contains atomic values.

Hence, it is already in **1NF**.

Second Normal Form (2NF)

Since the primary key is a single attribute (**StudentID**), there are no partial dependencies.

Hence, the relation is already in **2NF**.

Third Normal Form (3NF)

The following transitive dependencies exist:

- $\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{ClassName}$
- $\text{StudentID} \rightarrow \text{ClassID} \rightarrow \text{TeacherID} \rightarrow \text{TeacherName}$

These dependencies violate 3NF

BCNF Decomposition

STUDENT

Student(StudentID, StudentName, ClassID)

Primary Key: StudentID

CLASS

Class(ClassID, ClassName, TeacherID)

Primary Key: ClassID

TEACHER

Teacher(TeacherID, TeacherName)

Primary Key: TeacherID

SUBJECT

Subject(SubjectID, SubjectName)

Primary Key: SubjectID

Justification

Each determinant in every relation is a candidate key.

Therefore:

- No redundancy exists.
- No transitive dependency exists.
- Every relation satisfies BCNF.

Advantages of BCNF

- Eliminates redundancy.
- Removes update anomalies.
- Prevents insertion anomalies.
- Prevents deletion anomalies.
- Improves data consistency.
- Makes database maintenance easier.
- Enhances query performance.
- Ensures data integrity.

Conclusion

The School ERP database initially contains redundancy and transitive dependencies. By decomposing it into **STUDENT**, **CLASS**, **TEACHER**, and **SUBJECT** tables, every functional dependency satisfies the BCNF condition. This design improves consistency, minimizes redundancy, and ensures efficient database management.

Question 10

Case Study:

Given a movie streaming platform schema, identify **Join Dependencies (JD)** and decompose the relation into **Fifth Normal Form (5NF)** with **lossless-join verification**.

Introduction

A movie streaming platform stores details about movies, actors, directors, and subscriptions. When all information is stored in a single relation, complex join dependencies arise. Fifth Normal Form (5NF) eliminates redundancy caused by join dependencies while ensuring that decomposed relations can be joined back without any loss of information.

Assume the relation is:

Streaming(MovieID, MovieName, ActorID, ActorName, DirectorID, DirectorName, SubscriptionPlan)

Attribute Description

- **MovieID** – Movie identifier
- **MovieName** – Movie title
- **ActorID** – Actor identifier
- **ActorName** – Actor name
- **DirectorID** – Director identifier
- **DirectorName** – Director name
- **SubscriptionPlan** – User subscription plan

Functional Dependencies

1. $\text{MovieID} \rightarrow \text{MovieName}, \text{DirectorID}$
2. $\text{DirectorID} \rightarrow \text{DirectorName}$
3. $\text{ActorID} \rightarrow \text{ActorName}$

Join Dependencies (JD)

The following join dependencies exist:

- $\text{Movie} \leftrightarrow \text{Actor}$
- $\text{Movie} \leftrightarrow \text{Director}$
- $\text{Movie} \leftrightarrow \text{Subscription Plan}$

The information can be reconstructed only by joining multiple relations.

Normalization

1NF

All values are atomic.

2NF

No partial dependency exists.

3NF

Remove transitive dependency:

$\text{DirectorID} \rightarrow \text{DirectorName}$

BCNF

Every determinant becomes a candidate key.

Fifth Normal Form (5NF)

Decompose the relation into the following tables.

MOVIE

Movie(MovieID, MovieName, DirectorID)

Primary Key: MovieID

DIRECTOR

Director(DirectorID, DirectorName)

Primary Key: DirectorID

ACTOR

Actor(ActorID, ActorName)

Primary Key: ActorID

MOVIE_ACTOR

MovieActor(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

SUBSCRIPTION

Subscription(MovieID, SubscriptionPlan)

Primary Key:

(MovieID, SubscriptionPlan)

Lossless Join Verification

The decomposition is **lossless** because:

- Every table is connected through common primary and foreign keys.
- Joining **MOVIE**, **DIRECTOR**, **ACTOR**, **MOVIE_ACTOR**, and **SUBSCRIPTION** reconstructs the original relation exactly.
- No information is lost during decomposition.
- No spurious tuples are generated.

Therefore, the decomposition satisfies **5NF**.

Advantages of 5NF

- Eliminates join dependency redundancy.
- Ensures lossless joins.
- Improves consistency.
- Reduces duplication.
- Enhances scalability.
- Simplifies maintenance.
- Supports efficient database design.
- Improves integrity and reliability.

Conclusion

The movie streaming platform schema contains join dependencies that can lead to redundancy. By decomposing the relation into **MOVIE**, **DIRECTOR**, **ACTOR**, **MOVIE_ACTOR**, and **SUBSCRIPTION** tables, the database satisfies **Fifth Normal Form (5NF)**. The decomposition is lossless, preserves all information, and provides a highly efficient and reliable database design.